CSC 3320: System-Level Programming

Chapter 2, 3

Instructor: Dr. Md Mahfuzur Rahman
Department of Computer Science, GSU

Note: These slides are updated based on original notes from Raj Sunderraman, Michael Weeks, and Yuan Long

Constants

Integer Constants

- Decimal
  - 15 255 32767
- Octal
  - Must begin with zero
  - 017 03777 07777
- Hexadecimal
  - Must begin with zero and x
  - 0xf 0xff 0x7fff
- Long type
  - Put L (or l) at end
  - 15L 15l 0377L
- Unsigned type
  - Put U (or u) at end
  - 15U 15u 0x377U

Floating Constants

- Floating constants are stored as **double-precision** number.
  - 57.0 57. 57.0e0 .57e2
- Single precision
  - Put F (or f) at end
  - 57.0f 57. f 57.0e0F .57e2F
- Long type
  - Put L (or l) at end
  - 57.0L

Constants

Character and string constants

- Similar to Java
- A single character
  - ‘A’, ‘1’
  - Or return in terms of ASCII code
    - 0101 (Decimal code is 65, which is for ‘A’)
- A string literal
  - “text” , “I am a string”, “3/9”
- Special character
  - ‘**\n**’ = newline, ‘**\t**’= tab, ‘\\’ = backlash, ‘\”’ = double quotes

Declaring a variable

Similar to Java

Variables must be ***declared*** before they are used

Can be declared one at a time:

- int height;
- float profit;

Several can be declared at the same time:

- int height, length, width, volume;
- float profit, loss;

Assignment

Similar to Java

Assign a constant to a variable

- `profit = 2150.48;`
- `profit = 2150.48f;`
  - Note: Failing to include the `f` may cause a warning from the compiler.

Assign the value of an expression to a variable

```
height = 8;
length = 12;
width = 10;
volume = height * length * width;
```

Initialization

Similar to Java

```
int height, length = 12, width = 10;
```

Note: Some ***uninitialized*** variables are automatically set to zero when a program begins to execute, but most are not.

Attempting to access the value of an ***uninitialized*** variable may yield an unpredictable result.

What will be the output?

```c
#include <stdio.h>

int x;
int p;

int fun()
{
    static int a;
    int b;

    printf("%d\t%d", a, b);
}

int main()
{
    int y;

    printf("%d\t%d\n", x, p);
    fun();

    return 0;
}
```

Output:

```
0	0
0	31836
```

Reading and writing integers

unsigned integer

```
unsigned int u;
scanf("%u", &u);          /* reads u in base 10 */
printf("%u", u);          /* writes u in base 10 */
scanf("%o", &u);          /* reads u in base 8 */
printf("%o", u);          /* writes u in base 8 */
scanf("%x", &u);          /* reads u in base 16 */
printf("%x", u);          /* writes u in base 16*/
```

short integer

```
short int x;
scanf("%hd", &x);
printf("%hd", x);
```

long integer

```
long int x;
scanf("%ld", &x);
printf("%ld", x);
```

Reading and writing characters

A single character

```
char x;

scanf("%c", &x);
printf("%c", x);
```

A string

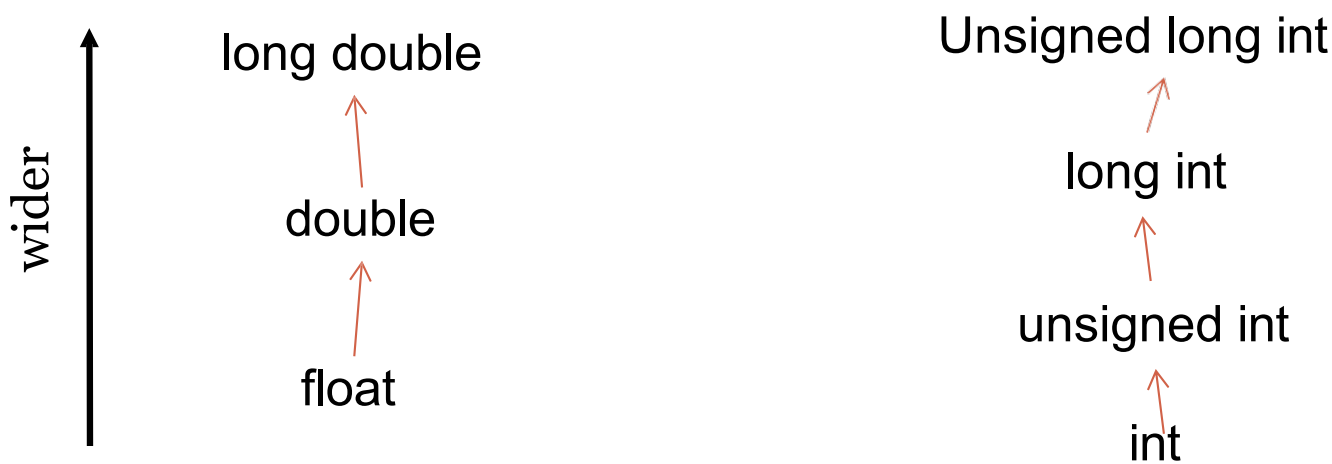
- You will learn it after introducing array and pointer

Type Conversions

Implicit conversion

- Narrower types are converted into wider types **automatically**

```
int a=1,d=0;
double b=2.55, c=0;
c= a + b; // a is converted to double
//But d= a + b; will cause data loss
```



Type Conversions

Explicit conversion

- The conversion is forced by **casts**
- But may loose precision
  - `float` to `int` conversion causes truncation of fractional part
  - `double` to `float` conversion causes rounding of digits
  - `long int` to `int` causes dropping of the higher order bits.

```
int a=1,d=0;
double b=2.55, c=0;
d= (int)(a + b); // a is converted to double first, then
a+b is casted to int.
```

Type conversion

More examples

```
float f, frac_part;
frac_part = f - (int) f;
```

```
float quotient;
int dividend, divisor;
quotient = (float) dividend / divisor;
```

```
short int i;
int j = 1000;
i = j * j;   /* WRONG */
```

User-defined types

`typedef` gives names to types

```
typedef int BOOL
BOOL flag;   /*  same as int flag; */

typedef shortint Int16
typedef long int Int32
typedef unsigned char Byte
Int16 a=0;
Int32 b=12;
Byte  c=1;

typedef struct {int age; char *name} person;
person people;
```

Chapter 3

What will be covered?

Formatted Input/output

Expressions

Bitwise Operations

Conditional Expressions

Control Flow

Formatted Input/Output

Read input: ***scanf*** function

- E.g. scanf("%d", &x);

Print output: ***printf*** function

- E.g. printf("%d", x);

Both *printf* function and *scanf* function must be supplied with a ***format string*** as 1st argument.

The format string may contain both ordinary characters and ***conversion specifications.***

- E.g. "x has a value %d, y has a value %f"
  - **%d** and **%f** are conversion specifications
  - Other characters are ordinary characters

print Function

Note: In following cases, the program has no compiling errors but will produce meaningless output.

- Too many or few conversion specifications
  - Example:

```
printf("%d %d\n", i);     /*** WRONG ***/
printf("%d\n", i, j);     /*** WRONG ***/
```

- Incorrect specification
  - Example:

```
printf("%f %d\n", i, x);   /*** WRONG ***/
```

the `int` variable `i` and the `float` variable `x` are in the wrong order

scanf Function

Note: The **&** is usually (but not always) required.

- No compiling errors if no & before variable name but may generate unexpected output.

Note: The number of conversion specifications should **match** the number of input variables.

- No compiling errors if not match, but may generate unexpected output.

How scanf work?

Sample input:

```
1-20.3-4.0e3¤
```

The call of `scanf` is the same as before:

```
scanf("%d%d%f%f", &i, &j, &x, &y);
```

Here's how `scanf` would process the new input:

- `%d`. Stores 1 into `i`.
- `%d`. Stores –20 into `j`.
- `%f`. Stores 0.3 into `x`.
- `%f`. Stores -4.0×10^3 into `y`.